Slide 1:
"Good morning, everyone. Today I will present the first half of my major project. I am building a system to solve hard optimization problems. Currently, I have finished the **Traveling Salesperson Problem**. In the future, I will add the **0/1 Knapsack Problem**."

Slide -2:
My project is divided into two phases. Phase 1 is about finding the shortest path between cities, which is done. Phase 2 will be about maximizing profit using the Knapsack algorithm, which is my next step. I used Java for the design and C language for speed.

Slide-3:
I started with the Traveling Salesperson Problem. It looks easy, but it is actually very hard. If you have just 12 cities, there are almost 500 million possible paths. If you try to check them all one by one, the computer will crash.

Slide-4:
To solve this, I first used the **Branch and Bound** algorithm. This method is perfect. It guarantees to find the absolute shortest path. It works by 'pruning' or cutting off bad paths early, so we don't waste time checking them.

Slide-5:
However, Branch and Bound is slow if we have 50 cities. So, I added a second algorithm called **Ant Colony Optimization**. This mimics how real ants find food using scent trails. It doesn't guarantee a perfect answer, but it is extremely fast for large maps.

Slide-6:
This is the most important part of my work. I compared both algorithms. For small maps, Branch and Bound is the winner because it is exact. But for large maps, Ant Colony is the winner because it doesn't crash. My system lets the user choose the best tool for the job

Slide-7:
As I mentioned, this is an ongoing project. My next immediate goal is to implement the **0/1 Knapsack Problem**. I will use **Dynamic Programming** to solve it. I will add this feature to the same C backend I have already built

Slide-8:
To conclude, I have successfully built a system that solves the TSP using two different methods. I have set up a strong foundation with Java and C, and I am ready to start the Knapsack implementation next. Thank you for listening